

TECHNICAL DOCUMENTATION v1.0

FitIntel Pro

ERP/CRM система управления фитнес-клубом
ERP/CRM System for Fitness Club Management

ООО «FixIntel»

Россия, г. Москва

Июнь 2026

Python 3.12 + FastAPI + PostgreSQL + React

Оглавление

1. Общее описание системы

- 1.1. Назначение и область применения
- 1.2. Технологический стек
- 1.3. Общая архитектура

2. Модули системы

- 2.1. Core (Auth, Users, RBAC)
- 2.2. CRM (Clients, Tariffs, Subscriptions)
- 2.3. Access Control (Credentials, Access, Lockers)
- 2.4. Visits
- 2.5. Finance (Wallet, Payments, Receipts, CashDesk, Sales)
- 2.6. Hardware Abstraction Layer
- 2.7. Analytics, Documents, Marketing, Gamification

3. API Reference

- 3.1. Auth API
- 3.2. Users API
- 3.3. CRM API
- 3.4. Finance API
- 3.5. Access Control API
- 3.6. Hardware API

4. Модели данных

- 4.1. ER-диаграмма
- 4.2. Описание таблиц

5. RBAC система

- 5.1. Роли
- 5.2. Разрешения

6. Hardware Abstraction Layer

- 6.1. Архитектура HAL
- 6.2. Подключение нового устройства

7. Frontend

- 7.1. Админ-панель
- 7.2. Личный кабинет клиента

8. Инфраструктура и деплой

- 8.1. Docker
- 8.2. CI/CD
- 8.3. Terraform (Yandex Cloud)
- 8.4. Мониторинг

9. Безопасность

- 9.1. Аутентификация и авторизация
- 9.2. Шифрование

10. Конфигурация

1. Общее описание системы

1.1. Назначение и область применения

FitIntel Pro (кодовое название проекта — FitNexus AI) представляет собой ERP/CRM-систему, разработанную специально для управления фитнес-клубами. Система обеспечивает полный цикл управления: от регистрации клиентов и продажи абонементов до контроля доступа на турникеты и аналитики посещаемости.

Таблица 1: Ключевые характеристики системы

Параметр	Значение
Версия	1.0
Язык разработки	Python 3.12
Веб-фреймворк	FastAPI 0.110+
База данных	PostgreSQL 16
Кэш / Брокер	Redis 7
ORM	SQLAlchemy 2.0
Миграции	Alembic
API эндпоинтов	168
SQLAlchemy моделей	26
Фронтенд	React 18 + TypeScript + Tailwind CSS
СКУД-драйверов	7 (ЭРА, X1, Kerong + Generic)

1.2. Технологический стек

Backend

Таблица 2: Компоненты бэкенда

Компонент	Версия	Назначение
FastAPI	0.110+	ASGI веб-фреймворк, OpenAPI/Swagger
Uvicorn	0.29+	ASGI сервер
SQLAlchemy	2.0+	ORM, модели, миграции
Psycopg	3.1+	PostgreSQL драйвер
Pydantic v2	2.6+	Валидация, сериализация, настройки
PyJWT	2.8+	JWT токены аутентификации
Passlib + Bcrypt	1.7+	Хеширование паролей
Celery	5.3+	Асинхронные задачи
Redis-py	5.0+	Кэш, сессии, брокер Celery
Alembic	1.13+	Миграции БД

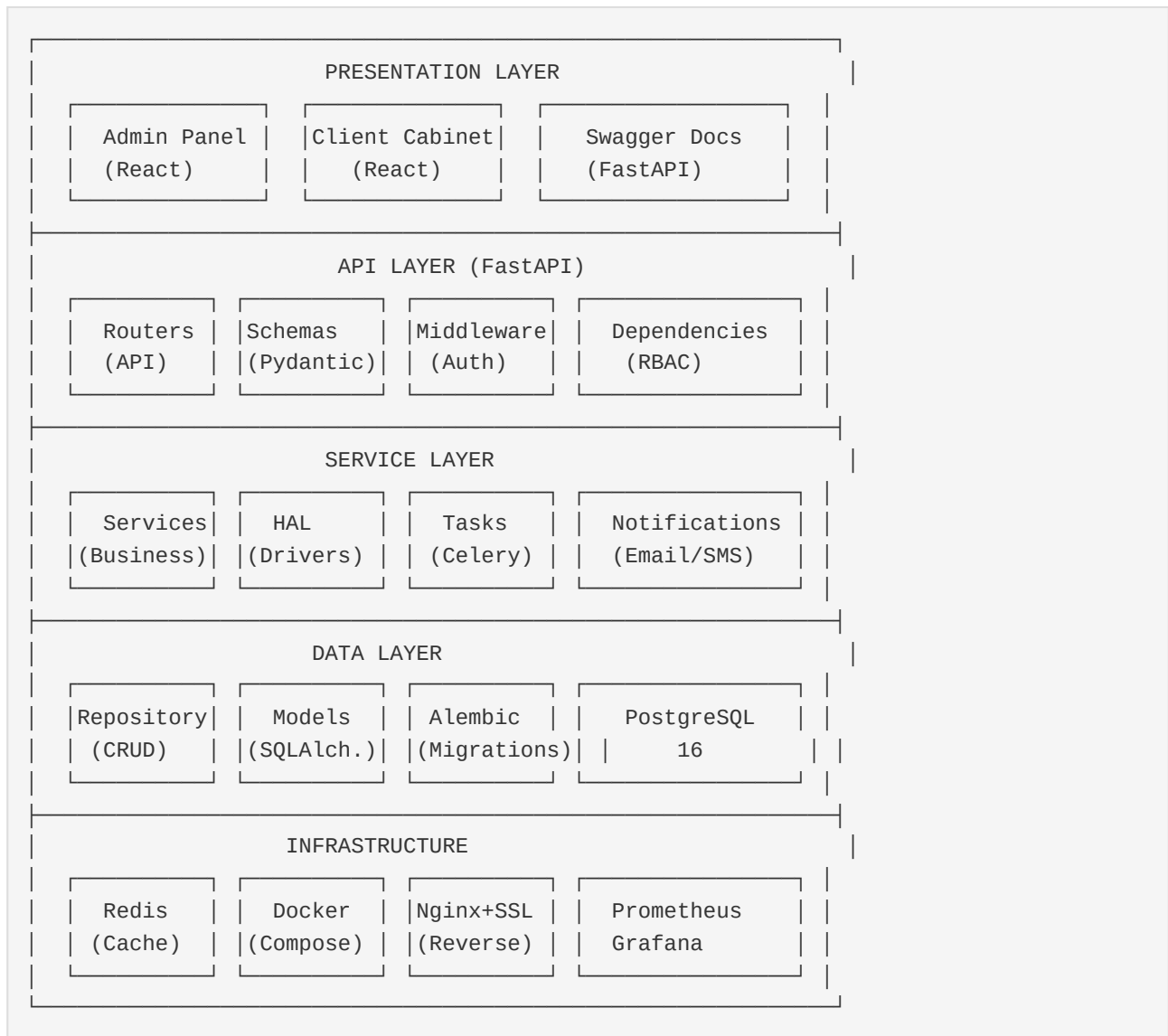
Frontend

Таблица 3: Компоненты фронтенда

Компонент	Версия	Назначение
React	18	UI фреймворк
TypeScript	5.4+	Типизация
Vite	5+	Сборщик
Tailwind CSS	3.4	Стилизация
React Router DOM	6	Маршрутизация
Axios	1.6+	HTTP клиент
Zustand	4	Управление состоянием
Lucide React	latest	Иконки

1.3. Общая архитектура

Система построена по принципу модульной многоуровневой архитектуры (Layered Architecture) с разделением ответственности (SoC):



Поток запроса: **Client** → **Nginx** → **FastAPI Router** → **Service** → **Repository** → **PostgreSQL**. Ответ идёт в обратном порядке.

2. Модули системы

2.1. Core (Auth, Users, RBAC)

Базовый модуль, обеспечивающий аутентификацию, управление пользователями и контроль доступа на уровне API.

Auth

- JWT-аутентификация с access token (Bearer)
- Двойная система логина: JSON API + OAuth2 (для Swagger)
- Токен expires через 30 минут (настраивается)
- Регистрация новых пользователей через Users API (требуется право `users.create`)

Users

- CRUD пользователей с фильтрацией и пагинацией
- Управление ролями пользователя (assign/revoke)
- Смена пароля, сброс пароля администратором
- Деактивация аккаунта

RBAC

- 9 системных ролей: owner, admin, manager, trainer, cashier, accountant, support, client, device
- 51 разрешение (permission) с точечной нотацией: `module.action`
- Матрица ролей и разрешений (/rbac/roles-matrix)
- Динамическая проверка прав на эндпоинтах

2.2. CRM (Clients, Tariffs, Subscriptions)

Clients

- Регистрация клиентов с валидацией телефона и email
- Статусы: ACTIVE, BLOCKED, FROZEN
- Категории: standard, vip, corporate, child, pensioner
- Timeline история событий клиента
- Поиск по ФИО, телефону, email

Tariffs

- Тарифные планы с периодом (month, quarter, half_year, year)
- Типы: visits (количество визитов) и period (безлимит по времени)

- Валюта, цена, архивация

Subscriptions

- Продажа абонементов по тарифам
- Статусы: ACTIVE, FROZEN, EXPIRED, CANCELLED, PENDING
- Жизненный цикл: freeze, unfreeze, renew, cancel
- История изменений (subscription_events)
- Автоматическая проверка срока действия

2.3. Access Control

Credentials (QR/RFID)

- Генерация QR-кодов для клиентов
- Создание RFID-меток
- Блокировка/разблокировка ключей
- Автоматическая регенерация QR при потере

Access

- Проверка доступа по QR/RFID
- Принудительный доступ (override)
- Кэширование прав доступа в Redis
- Логирование всех попыток прохода

Lockers

- Управление шкафчиками через HAL
- Освобождение шкафчика (release)
- Проверка статуса (занят/свободен)

2.4. Visits

- Фиксация входа (entry) и выхода (exit)
- Ручная фиксация посещения (manual)
- Автоматическое завершение (complete) по таймауту
- Статистика: today, week, month
- Среднее время посещения
- Список клиентов в зале (active)

2.5. Finance

Wallet

- Личный кошелёк клиента
- Пополнение (deposit)
- Заморозка средств (frozen_balance)
- История транзакций

Payments

- Создание платежей
- Статусы: PENDING, COMPLETED, FAILED, REFUNDED
- Возврат (refund)
- Онлайн-оплата
- Webhooks от платёжных систем

Receipts

- Фискальные чеки
- Отправка по email
- Генерация PDF
- Повторная отправка в ОФД

Cash Desk

- Открытие/закрытие смены
- Кассовые операции (приход/расход)
- Сверка кассы (verify)
- Отчёт по смене

Sales

- Продажа абонементов
- Продажа разовых услуг
- Пакетные продажи

2.6. Hardware Abstraction Layer

HAL обеспечивает универсальный интерфейс для работы с ЛЮБЫМИ СКУД-устройствами. Архитектура основана на паттерне Strategy + Registry.

```

BaseDeviceDriver (ABC)
├─ EraDriver          → ЭРА (TCP бинарный протокол)
├─ X1Driver           → X1 (HTTP REST API + WebSocket)
├─ KerongDriver       → Kerong (Modbus RTU over TCP)
├─ GenericModbusDriver → Любой Modbus TCP/RTU
├─ GenericHttpDriver  → Любой HTTP API
└─ GenericMqttDriver  → Любой MQTT

```

Добавление нового устройства происходит через YAML-конфиг `config/devices.yaml` без изменения кода приложения.

2.7. Дополнительные модули

Таблица 4: Дополнительные модули

Модуль	Описание	Статус
Analytics	Дашборд, аналитика посещений, финансовый анализ, retention	Реализован
Documents	Шаблоны договоров, генерация документов, скачивание PDF	Реализован
Marketing	Маркетинговые кампании, сегменты клиентов, SMS-рассылка	Реализован
Gamification	Уровни клиентов, достижения, баллы	Реализован
Online Training	Онлайн-тренировки, сессии	Заглушка
Devices	Управление устройствами (ping, статус)	Реализован

3. API Reference

3.1. Auth API

Таблица 5: Auth Endpoints

Метод	Endpoint	Описание
POST	/api/v1/auth/login	JWT логин (JSON: login + password)
POST	/api/v1/auth/token	OAuth2 логин (form: username + password)
GET	/api/v1/auth/me	Текущий пользователь
GET	/api/v1/auth/token-check	Проверка токена
GET	/api/v1/auth/permissions/users-read-single	Проверка права users.read
POST	/api/v1/auth/permissions/users-create-and-read	Проверка прав users.read + create
GET	/api/v1/auth/roles/admin-or-owner	Проверка роли admin/owner

3.2. Users API

Таблица 6: Users Endpoints (14)

Метод	Endpoint	Описание
GET	/api/v1/users/me	Свой профиль
PATCH	/api/v1/users/me/update	Обновить профиль
POST	/api/v1/users/me/change-password	Сменить пароль
GET	/api/v1/users/	Список (offset, limit, search)
GET	/api/v1/users/{user_id}	Пользователь по UUID
POST	/api/v1/users/	Создать пользователя
PATCH	/api/v1/users/{user_id}	Обновить пользователя
POST	/api/v1/users/{user_id}/deactivate	Деактивировать
POST	/api/v1/users/{user_id}/reset-password	Сбросить пароль
POST	/api/v1/users/{user_id}/roles/assign	Назначить роль
POST	/api/v1/users/{user_id}/roles/revoke	Снять роль
GET	/api/v1/users/{user_id}/roles	Роли пользователя
GET	/api/v1/users/{user_id}/permissions	Права пользователя

3.3. CRM API

Таблица 7: CRM Endpoints

Метод	Endpoint	Описание
GET	/api/v1/clients/	Список клиентов
GET	/api/v1/clients/{client_id}	Клиент по UUID
POST	/api/v1/clients/	Создать клиента
PATCH	/api/v1/clients/{client_id}	Обновить клиента
GET	/api/v1/clients/{client_id}/timeline	История клиента
GET	/api/v1/tariffs/	Список тарифов
POST	/api/v1/tariffs/	Создать тариф
GET	/api/v1/subscriptions/	Список абонементов
POST	/api/v1/subscriptions/	Создать абонемент
POST	/api/v1/subscriptions/{id}/freeze	Заморозить
POST	/api/v1/subscriptions/{id}/unfreeze	Разморозить
POST	/api/v1/subscriptions/{id}/renew	Продлить
POST	/api/v1/subscriptions/{id}/cancel	Отменить
GET	/api/v1/subscriptions/{id}/history	История изменений

3.4. Finance API

Таблица 8: Finance Endpoints

Метод	Endpoint	Описание
GET	/api/v1/wallet/me	Мой кошелёк
GET	/api/v1/wallet/me/balance	Баланс
GET	/api/v1/wallet/me/transactions	Транзакции
POST	/api/v1/wallet/client/{id}/deposit	Пополнение
GET	/api/v1/payments/me	Мои платежи
POST	/api/v1/payments/	Создать платёж
POST	/api/v1/payments/{id}/complete	Завершить
POST	/api/v1/payments/{id}/refund	Возврат
GET	/api/v1/receipts/{receipt_id}	Чек
GET	/api/v1/receipts/{id}/pdf	PDF чека
POST	/api/v1/cash-desk/open	Открыть смену
POST	/api/v1/cash-desk/close	Закрыть смену
GET	/api/v1/cash-desk/current	Текущая смена
POST	/api/v1/sales/subscription	Продажа абонемента
POST	/api/v1/sales/service	Продажа услуги

3.5. Access Control API

Таблица 9: Access Control Endpoints

Метод	Endpoint	Описание
POST	/api/v1/access/check	Проверить доступ по credential
POST	/api/v1/access/grant	Предоставить доступ
POST	/api/v1/access/exit	Фиксация выхода
POST	/api/v1/access/override	Принудительный доступ
POST	/api/v1/credentials/qr	Создать QR-код
GET	/api/v1/credentials/qr/client/{id}	Получить QR клиента
POST	/api/v1/credentials/qr/client/{id}/regenerate	Перегенерировать QR
POST	/api/v1/credentials/rfid	Создать RFID
POST	/api/v1/credentials/{id}/block	Заблокировать ключ
POST	/api/v1/credentials/{id}/unblock	Разблокировать ключ
POST	/api/v1/lockers/release	Освободить шкафчик
GET	/api/v1/lockers/status/{id}	Статус шкафчика

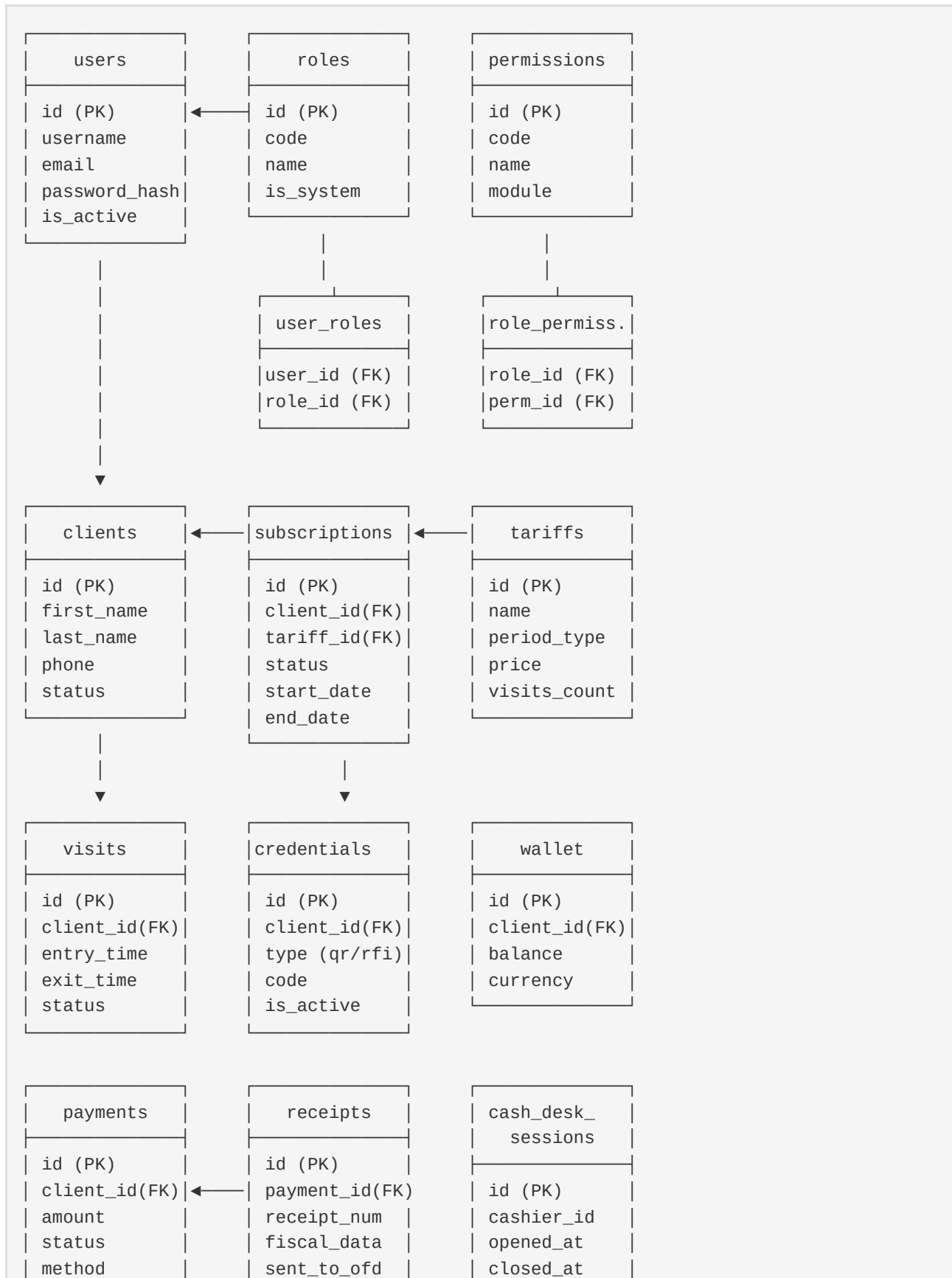
3.6. Hardware API

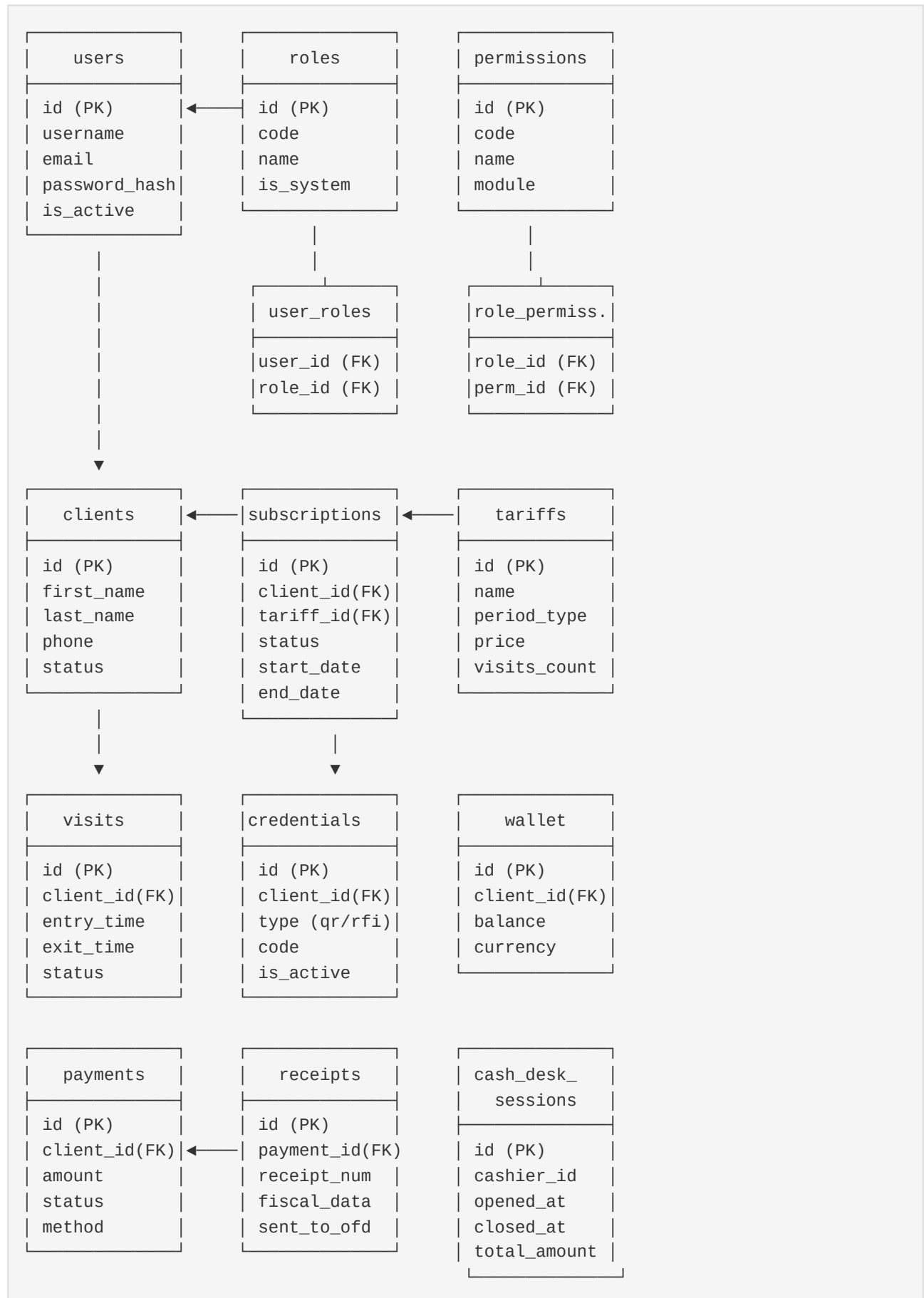
Таблица 10: Hardware Endpoints

Метод	Endpoint	Описание
GET	/api/v1/hardware/devices	Список устройств
GET	/api/v1/hardware/devices/{id}	Инфо об устройстве
POST	/api/v1/hardware/devices/{id}/open	Открыть
POST	/api/v1/hardware/devices/{id}/close	Закрыть
POST	/api/v1/hardware/devices/{id}/ping	Ping
GET	/api/v1/hardware/devices/{id}/info	Подробная инфо
POST	/api/v1/hardware/devices/{dev}/lockers/{id}/release	Открыть шкафчик
GET	/api/v1/hardware/devices/{dev}/lockers/{id}/status	Статус шкафчика
GET	/api/v1/hardware/drivers	Список драйверов
POST	/api/v1/hardware/devices	Добавить устройство
DELETE	/api/v1/hardware/devices/{id}	Удалить устройство

4. Модели данных

4.1. ER-диаграмма (основные сущности)





4.2. Описание таблиц

Таблица 11: Основные таблицы базы данных

Таблица	Назначение	Ключевые поля
users	Пользователи системы	id, username, email, password_hash, is_active
roles	Роли (RBAC)	id, code, name, is_system
permissions	Разрешения (RBAC)	id, code, name, module
user_roles	Связь пользователей и ролей	user_id, role_id
role_permissions	Связь ролей и разрешений	role_id, permission_id
clients	Клиенты фитнес-клуба	id, first_name, last_name, phone, email, status
tariffs	Тарифные планы	id, name, period_type, price, visits_count
subscriptions	Абонементы клиентов	id, client_id, tariff_id, status, start_date, end_date
subscription_events	История событий абонемента	id, subscription_id, event_type, created_at
visits	Посещения	id, client_id, entry_time, exit_time, duration_minutes, status
credentials	QR/RFID ключи доступа	id, client_id, type, code, is_active
access_logs	Логи доступа	id, credential_id, device_id, result, timestamp
wallets	Кошельки клиентов	id, client_id, balance, frozen_balance, currency
wallet_transactions	Транзакции кошелька	id, wallet_id, type, amount, description
payments	Платежи	id, client_id, amount, status, method, receipt_id
receipts	Фискальные чеки	id, payment_id, receipt_number, fiscal_data, sent_to_ofd
cash_desk_sessions	Кассовые смены	id, cashier_id, opened_at, closed_at, status
cash_operations	Кассовые операции	id, session_id, type, amount, description
devices	СКУД-устройства	id, name, vendor, model, device_type, ip, status

audit_logs Аудит действий id, user_id, action, entity_type, entity_id, timestamp

5. RBAC система

5.1. Роли

Таблица 12: Системные роли

Роль	Код	Описание
Владелец	owner	Полный доступ ко всем функциям
Администратор	admin	Управление пользователями, настройки
Менеджер	manager	CRM, финансы, отчёты
Тренер	trainer	Клиенты, посещения, график
Кассир	cashier	Продажи, касса, чеки
Бухгалтер	accountant	Финансы, отчёты, сверка
Поддержка	support	Просмотр данных, базовые операции
Клиент	client	Личный кабинет (self-service)
Устройство	device	API-ключ для СКУД-устройств

5.2. Разрешения

51 разрешение с точечной нотацией `module.action`. Примеры:

Таблица 13: Примеры разрешений

Разрешение	Описание
users.read	Просмотр пользователей
users.create	Создание пользователей
users.update	Редактирование пользователей
users.delete	Удаление пользователей
clients.read	Просмотр клиентов
clients.create	Создание клиентов
subscriptions.manage	Управление абонементом
payments.read	Просмотр платежей
payments.create	Создание платежей
devices.read	Просмотр устройств
devices.update	Управление устройствами
analytics.read	Просмотр аналитики
cash_desk.open	Открытие кассовой смены
cash_desk.close	Закрытие кассовой смены

Проверка прав на эндпоинтах выполняется через зависимости FastAPI: `Depends(require_permission("users.read"))`. Пользователи с ролями `owner` и `admin` имеют доступ ко всем эндпоинтам автоматически.

6. Hardware Abstraction Layer

6.1. Архитектура HAL

HAL реализован по паттерну **Strategy** + **Registry**. Базовый класс `BaseDeviceDriver` определяет интерфейс, который должны реализовать все драйверы. `Registry` автоматически обнаруживает и регистрирует драйверы из папки `app/hardware/drivers/`.

DeviceManager (центральный менеджер, singleton)				
DriverRegistry (автообнаружение + фабрика драйверов)				
EraDriver	X1Driver	Kerong	Generic	Generic
TCP	HTTP+WS	Driver	Modbus	Http/Mqtt
бинарный	REST API	Modbus	Driver	Driver
		RTU/TCP	TCP/RTU	REST/MQTT

Интерфейс BaseDeviceDriver

Таблица 14: Методы BaseDeviceDriver

Метод	Назначение
connect()	Подключиться к устройству
disconnect()	Отключиться
ping()	Проверка связи
open(credential, **kwargs)	Открыть проход
close(**kwargs)	Закрыть/блокировать
release_locker(locker_id)	Открыть шкафчик
get_locker_status(locker_id)	Статус шкафчика
get_device_info()	Информация об устройстве
sync_credentials(credentials)	Синхронизация ключей
start_event_listener()	Прослушка событий

6.2. Подключение нового устройства

Способ 1: YAML-конфиг (без кода)

Отредактируй `config/devices.yaml`:

```
- device_id: "my_turnstile_01"
  name: "Турникет — Новый"
  vendor: "Generic"
  model: "HTTP-API"
  device_type: "turnstile"
  protocol: "http_api"
  host: "192.168.1.200"
  port: 80
  endpoints:
    open: {method: "POST", path: "/api/open"}
    close: {method: "POST", path: "/api/close"}
    status: {method: "GET", path: "/api/status"}
```

Способ 2: Новый драйвер (для сложных протоколов)

```
from app.hardware.base import BaseDeviceDriver, register_driver, AccessResult

@register_driver
class MyDeviceDriver(BaseDeviceDriver):
    VENDOR = "MyCompany"
    MODELS = ["MODEL-A"]

    async def connect(self) -> bool: ...
    async def open(self, credential=None, **kwargs) -> AccessResult: ...
    async def close(self, **kwargs) -> bool: ...
```

7. Frontend

7.1. Админ-панель

Таблица 15: Страницы админ-панели

Страница	Путь	Функционал
Дашборд	/	KPI-карточки, быстрые действия, статус системы
Клиенты	/clients	Список, поиск, создание (dialog)
Абонементы	/subscriptions	Карточки, freeze/unfreeze/renew/cancel
Посещения	/visits	Вход/выход, активные в зале, статистика
Финансы	/finance	Кошелёк, платежи, чеки (табы)
Доступ	/access	Проверка QR/RFID, управление ключами
Устройства	/devices	Статус online/offline, heartbeat
Аналитика	/analytics	График посещений по дням
Настройки	/settings	Профиль, роли, права, системная инфо

7.2. Личный кабинет клиента

Таблица 16: Страницы клиентского кабинета

Страница	Путь	Функционал
Профиль	/cabinet	Данные клиента, баланс, кол-во абонементов
Абонементы	/cabinet/subscriptions	Список со статусами и датами
Посещения	/cabinet/visits	История с длительностью
Кошелёк	/cabinet/wallet	Баланс, история транзакций

8. Инфраструктура и деплой

8.1. Docker

Таблица 17: Docker-конфигурация

Файл	Назначение
Dockerfile	Production образ Python + FastAPI
docker-compose.yml	Dev: PostgreSQL + Redis
deploy/docker-compose.dev.yml	Полный dev stack (API + DB + Redis)
deploy/docker-compose.prod.yml	Production: API + DB + Redis + Nginx + Celery

8.2. CI/CD

GitLab CI pipeline (.gitlab-ci.yml):

```
lint → test → build → deploy
```

8.3. Terraform (Yandex Cloud)

Таблица 18: Terraform модули

Модуль	Ресурсы
network	VPC, подсети, security groups
compute	Виртуальные машины (API, DB, Redis)
alb	Application Load Balancer

8.4. Мониторинг

Таблица 19: Компоненты мониторинга

Компонент	Назначение
Prometheus	Сбор метрик
Grafana	Визуализация дашбордов
Zabbix	Мониторинг инфраструктуры
Alertmanager	Уведомления о проблемах

9. Безопасность

9.1. Аутентификация и авторизация

- **JWT** — Bearer токены с TTL 30 минут
- **OAuth2** — для Swagger UI (flow: password)
- **RBAC** — 9 ролей, 51 разрешение
- **Защита эндпоинтов** — через FastAPI Depends
- **Админ/владелец** — полный доступ ко всем API

9.2. Шифрование

- **Пароли** — Bcrypt (passlib), 12 rounds
- **JWT** — HS256, секрет из переменной окружения SECRET_KEY
- **PostgreSQL** — SSL в production
- **Nginx** — TLS 1.2/1.3, rate limiting

10. Конфигурация

Система использует переменные окружения (.env) + Pydantic Settings:

Таблица 20: Переменные окружения

Переменная	Описание	Пример
APP_ENV	Среда (dev/prod)	dev
APP_DEBUG	Режим отладки	false
SECRET_KEY	JWT секрет	random_string_32+
DATABASE_URL	PostgreSQL DSN	postgresql+psycopg://...
REDIS_HOST	Redis сервер	localhost
REDIS_PORT	Redis порт	6379
ACCESS_TOKEN_EXPIRE_MINUTES	TTL токена	30
DOCS_ENABLED	Swagger/ReDoc	true
SENTRY_DSN	Мониторинг ошибок	https://...

Файл `.env.example` содержит все переменные с примерами значений. Для production скопируйте в `.env` и заполните реальными значениями.